Intermediate Python

Naeemullah Khan

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

LMH
Lady Margaret Hall
University of Oxford

KAUST Academy
King Abdullah University of Science and Technology
Stage 1 · Course 2

June 13, 2026

Module 1
FUNCTIONS

def, arguments, scope, recursion, and lambda

Module Content

This module covers designing reusable, well-documented functions — including scope, recursion, first-class behaviour, and lambda expressions.

- **Functions**: using `def`, function calls, and program flow.
- **Arguments**: positional, keyword, and default parameter values.
- **Return values**: `return`, `print`, and side effects.
- **Docstrings**: documenting functions clearly.
- **Scope**: the LEGB rule, local/global variables, `global` and `nonlocal`.
- **Pure functions** vs. side effects.
- **Recursion**: factorial, Fibonacci, and recursive thinking.
- **First-class functions**: passing functions and lambda expressions.

Functions as Black Boxes

A **function** is a named sequence of instructions. You give it inputs, it performs a task, and it may return a result.


```
price = round(6.8275, 2)
print(price)      # 6.83
```

Key idea

You do not need to know how `round()` is implemented. You only need to know what arguments it expects and what result it returns.

Defining Functions with def

We define a function with `def`. The function body is indented, just like `if` statements and loops.

```
def cube_volume(side_length):
    volume = side_length ** 3
    return volume

print(cube_volume(2))   # 8
```

Vocabulary

- **Function name**: `cube_volume`
- **Parameter**: `side_length`, the name used inside the function
- **Argument**: 2, the value passed when calling the function
- **return**: sends a result back to the caller

Function Calls and Program Flow

A function definition is remembered by Python, but the body does not run until the function is called.

```
def area(width, height):
    return width * height

room = area(4, 6)
print(room)
```

Execution order

1. Python stores the function definition.
2. Python reaches `area(4, 6)`.
3. Parameters receive the arguments.
4. The function returns 24.
5. The caller stores it in `room`.

Positional vs. Keyword Arguments

Arguments can be matched by **position** or by **name**.

```
def describe_student(name, major, year):
    return f"{name} studies {major} in year {year}."

# Positional: order matters
print(describe_student("Aisha", "AI", 2))

# Keyword: names make the call clearer
print(describe_student(name="Aisha", major="AI", year=2))
print(describe_student(year=2, name="Aisha", major="AI"))
```

Beginner habit

Use keyword arguments when many numbers or similar-looking values are passed. They make code easier to read.

Default Parameter Values

A default parameter value is used when the caller does not provide that argument.

```
def greet(name, language="English"):
    if language == "Arabic":
        return f"Ahlan, {name}!"
    return f"Hello, {name}!"

print(greet("Sara"))                    # uses default
print(greet("Sara", language="Arabic"))
```

Rule

Parameters with default values should usually come after parameters without defaults.

return, print, and Side Effects

A function can **return** a value, or it can do something visible such as printing or writing a file.

Returns a value

```
def add(a, b):
    return a + b

x = add(2, 3)
```

Good for calculations and reuse.

Has a side effect

```
def print_total(a, b):
    print(a + b)

print_total(2, 3)
```

Good when the task is output.

Docstrings: Documenting Functions

A **docstring** explains what a function does. It is written as a string immediately under the function header.

```
def rectangle_area(width, height):
    """Return the area of a rectangle.

    width and height must be non-negative numbers.
    """
    return width * height
```

Good docstrings answer

- What does this function do?
- What do the parameters mean?
- What does it return?
- Are there assumptions or special cases?

Variable Scope: The LEGB Rule

When Python sees a name, it searches for it in a specific order called **LEGB**.

Local: inside the current function

Enclosing: outer function, if nested

Global: module-level names

Built-in: names like `len`, `print`, `max`

Key idea

A variable created inside a function is normally local to that function.

Local and Global Variables

A local variable lives inside a function. A global variable is defined outside functions.

```
message = "global message"

def show_message():
    message = "local message"
    print("Inside:", message)

show_message()
print("Outside:", message)
```

What happens?

The local `message` inside the function does not replace the global `message`. They are two different variables with the same name in different scopes.

global and nonlocal

Use `global` to update a module-level variable. Use `nonlocal` to update a variable from an enclosing function.

```
count = 0

def increment():
    global count
    count += 1
```

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment
```

Beginner warning

These keywords are useful, but frequent use can make programs harder to understand. Prefer passing values and returning results when possible.

Pure Functions vs. Side Effects

A **pure function** depends only on its inputs and returns a result without changing the outside world.

Pure function

```
def total_price(price, tax):
    return price + price * tax
```

Same input gives same output.

Side effect

```
items = []

def add_item(item):
    items.append(item)
```

Changes something outside itself.

Why this matters

Pure functions are easier to test. Side effects are sometimes necessary, but they should be intentional.

Recursion: A Function Calling Itself

Recursion solves a problem by reducing it to a smaller version of the same problem.

Every recursive function needs:

- a **base case**: when to stop,
- a **recursive case**: how to make progress,
- trust that the smaller call returns the right result.

Call stack idea

Each function call waits for the smaller call to finish. Python remembers these waiting calls in the **call stack**.

Recursive Example: Factorial

Factorial means multiplying a number by all positive integers below it.

```python
def factorial(n):
    """Return n! for a non-negative integer n."""
    if n == 0:
        return 1                  # base case
    return n * factorial(n - 1)   # recursive case

print(factorial(5))               # 120
```

Trace

`factorial(5)` becomes `5 * factorial(4)`, then `4 * factorial(3)`, and so on until `factorial(0)` returns 1.

Recursive Example: Fibonacci

The Fibonacci sequence starts with 0 and 1. Each next value is the sum of the previous two values.

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Important lesson

This definition is mathematically clear, but it repeats a lot of work. It is useful for learning recursion, not always for performance.

Functions Are First-Class Objects

In Python, functions are values. You can store them in variables, pass them to other functions, and return them.

```
def shout(text):
    return text.upper()

make_loud = shout
print(make_loud("hello"))
```

Why this matters

This idea enables callbacks, sorting keys, decorators, event handlers, and many library patterns.

Passing Functions as Arguments

A function that receives another function is called a **higher-order function**.

```
def apply_twice(func, value):
    return func(func(value))

def add_one(x):
    return x + 1

print(apply_twice(add_one, 10))   # 12
```

Concept

`apply_twice` does not know what `func` does. It only knows it can call it.

Lambda Expressions

A **lambda** is a small anonymous function. Use it for short operations, not large logic.

```
square = lambda x: x * x
add_tax = lambda price: price * 1.15

print(square(5))
print(add_tax(100))
```

Rule of thumb

If a lambda becomes hard to read, write a normal `def` function instead.

Module 2
STRINGS & TEXT PROCESSING

String methods, formatting, and Unicode

Module Content

This module covers Python's rich set of string tools for text manipulation, formatting, and Unicode support.

- **String basics**: text values, quotes, and immutability.
- **Common string methods**: upper, lower, strip, replace, find, and more.
- **Splitting and joining**: tokenizing text and reassembling sequences.
- **String formatting**: using `.format()` for dynamic output.
- **Unicode**: multilingual text support and character encoding.

Strings Are Text Values

A **string** is a sequence of characters surrounded by quotes.

```
name = "Aisha"
course = 'Python'
message = "Welcome to Module 3"
```

Key idea

Strings can store words, sentences, symbols, numbers written as text, Arabic text, emojis, file names, and user input.

String indexing

```
word = "Python"
print(word[0])   # P
print(word[1])   # y
```

String length

```
word = "Python"
print(len(word)) # 6
```

String Immutability

Strings are **immutable**: after a string is created, its characters cannot be changed in place.

This does not work

```
word = "Python"
word[0] = "J"      # TypeError
```

Create a new string

```
word = "Python"
new_word = "J" + word[1:]
print(new_word)    # Jython
```

Beginner takeaway

String methods usually **return a new string**. They do not modify the original string.

```
text = "  python  "
clean = text.strip()
print(text)    # original still has spaces
print(clean)   # new cleaned string
```

Common String Methods

String methods are functions attached to string objects. They help clean and transform text.

Method	Purpose	Example idea
`upper()`, `lower()`	Change letter case	normalize names or commands
`strip()`	Remove surrounding spaces	clean user input
`split()`	Break text into pieces	words or CSV-like text
`join()`	Combine pieces into text	build a sentence or path
`replace()`	Replace matching text	clean symbols or typos
`find()`	Locate a substring	search inside text
`startswith()`, `endswith()`	Check prefix/suffix	file names, IDs, URLs

```
raw = "  Python, AI, Data  "
text = raw.strip().lower()
parts = text.split(",")
print(parts)      # ['python', ' ai', ' data']
```

Habit

When processing user input, clean it first: remove spaces, normalize case, then compare or analyze.

Splitting, Joining, and Tokenization

Tokenization means breaking text into meaningful pieces called tokens. For beginners, a token is often just a word.

Split text

```
sentence = "AI helps science"
words = sentence.split()
print(words)
# ['AI', 'helps', 'science']
```

Join text

```
words = ["AI", "helps", "science"]
text = " ".join(words)
print(text)
# AI helps science
```

NLP preview

Many language-processing tasks begin by cleaning text, splitting it into tokens, and counting or comparing those tokens.

String Formatting with `.format()`

The **.format()** method places values inside placeholders written as {}. It is useful for building clear messages from variables.

Basic placeholders

```
name = "Sara"
score = 92
msg = "{} scored {}".format(name, score)
print(msg)
# Sara scored 92
```

Number formatting

```
price = 19.987
print("Price: {:.2f}".format(price))
# Price: 19.99
```

Named placeholders

```
template = "Hello {student}, welcome to {course}."
print(template.format(student="Ali", course="Python"))
```

Note: f-strings are often shorter, but `.format()` is still common in older code and reusable templates.

Unicode and Multilingual Text

Modern programs must handle accented and multilingual text. Python strings are Unicode text.

```
text = "Cafe Python"
print(len(text))

encoded = text.encode("utf-8")
decoded = encoded.decode("utf-8")
print(decoded)
```

Unicode

A standard way to represent characters from many writing systems.

UTF-8

A common encoding that stores Unicode text as bytes in files and networks.

Practical habit

When opening text files, specify the encoding when needed: `open("file.txt", encoding="utf-8")`.

Module 3
COLLECTIONS

Lists, tuples, dictionaries, and sets

Module Content

This module covers Python's four core collection types for organising and managing groups of data.

- **Lists**: creation, indexing, slicing, mutation, and common methods.
- **List copying**: shallow vs. deep copy.
- **Tuples**: immutable ordered data and use cases.
- **Dictionaries**: key-value storage, common methods, and nested dicts.
- **Sets**: unique values and set operations.
- **Choosing**: picking the right data structure for the problem.

Why Collections Matter

Key idea

A collection stores many related values under one name, instead of creating many separate variables.

```
score1 = 90
score2 = 85
score3 = 92
```

```
scores = [90, 85, 92]
```

- Collections make repeated processing easier.
- They work naturally with loops and functions.
- Different collections are useful for different tasks.

Lists: Creation, Indexing, and Slicing

```
grades = [95, 82, 77, 90]

first = grades[0]      # 95
last = grades[-1]      # 90
middle = grades[1:3]   # [82, 77]
```

Beginner rule

Python indexes start at 0. A slice `a:b` starts at `a` and stops before `b`.

- Lists preserve order.
- Lists can contain numbers, strings, booleans, and even other lists.

Lists Are Mutable

Mutable means changeable

A list can be edited after it is created.

```
items = ["pen", "book", "bag"]
items[0] = "pencil"
items.append("laptop")

print(items)
# ['pencil', 'book', 'bag', 'laptop']
```

- This is different from strings, which cannot be changed character by character.
- Nested lists can represent tables, grids, or grouped data.

Common List Methods

Method	Purpose
`append(x)`	Add one item to the end.
`extend(seq)`	Add many items from another iterable.
`insert(i,x)`	Insert an item at a position.
`remove(x)`	Remove the first matching value.
`pop(i)`	Remove and return an item by index; default is last item.
`sort()`	Sort the list in place.
`sorted(seq)`	Return a new sorted list.
`reverse()`	Reverse the list in place.
`index(x)`	Find the first position of a value.
`count(x)`	Count occurrences of a value.

List Copying: Shallow vs. Deep Copy

```
a = [[1, 2], [3, 4]]
b = a          # same list object
c = a.copy()   # shallow copy
```

Important distinction

A shallow copy creates a new outer list, but nested objects may still be shared.

```
import copy

d = copy.deepcopy(a)  # copies nested objects too
```

- Use `copy()` or slicing for simple one-level lists.
- Use `copy.deepcopy()` when nested structures must be fully independent.

Tuples: Immutable Ordered Data

```
point = (3, 5)
month, day, year = (5, 8, 2026)
```

- A tuple is like a list, but it cannot be changed after creation.
- Use tuples for fixed records: coordinates, dates, RGB colors, function results.

Named tuples

A named tuple gives names to positions, making tuple data easier to read.

```
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 5)
print(p.x, p.y)
```

Dictionaries: Key–Value Data

Key idea

A dictionary maps each unique key to a value.

```
student = {
    "name": "Aisha",
    "major": "AI",
    "year": 2
}

print(student["name"])
student["year"] = 3
```

- Keys are used instead of numeric indexes.
- Values can be numbers, strings, booleans, or other collections.

Common Dictionary Methods

Method	Purpose
`get(key, default)`	Safely read a value without raising `KeyError`.
`keys()`	View all keys.
`values()`	View all values.
`items()`	View key–value pairs.
`update(other)`	Add or replace multiple pairs.
`pop(key)`	Remove a key and return its value.
`setdefault(k, v)`	Return existing value, or create it if missing.

Beginner habit

Use `get()` when a key may not exist.

Nested Dictionaries and Dict Comprehensions

```
students = {
    "Aisha": {"score": 95, "level": "A"},
    "Omar":  {"score": 88, "level": "B"}
}

print(students["Aisha"]["score"])
```

Dictionary comprehension

Build a dictionary from a pattern.

```
squares = {n: n*n for n in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Sets: Unique Values and Set Operations

```python
colors = {"red", "green", "blue", "red"}
print(colors)   # duplicates are removed
```

Operation		Meaning
`a | b`	**or** `a.union(b)`	Union: values in either set.
`a & b`	**or** `a.intersection(b)`	Intersection: values in both sets.
`a - b`	**or** `a.difference(b)`	Difference: values in `a` not in `b`.
`a ^ b`		Symmetric difference: values in exactly one set.

Choosing the Right Data Structure

Structure	Best for	Main trade-off
List	Ordered sequence, indexing, repeated values	Searching can be slower for large data.
Tuple	Fixed ordered record	Cannot be changed.
Dictionary	Fast lookup by key	Requires meaningful unique keys.
Set	Fast membership testing and unique values	No indexing or guaranteed position-based access.

Performance intuition

Use a set or dictionary when you repeatedly ask: "Is this value present?" or "What value belongs to this key?"

Module 4

COMPREHENSIONS & FUNCTIONAL TOOLS

Comprehensions, generators, and functional tools

Module Content

This module introduces compact, expressive Python patterns using comprehensions, generators, and functional tools.

- **Comprehensions**: list, dictionary, set, and generator expressions.
- **Nested comprehensions**: multi-level collection building.
- **Generators**: lazy evaluation with generator expressions.
- **Anti-patterns**: avoiding mutation while iterating.
- **Functional tools**: `map()`, `filter()`, `zip()`, `enumerate()`, and `sorted(key=...)`.
- **Variable-length arguments**: `*args` and `**kwargs`.
- **Functional vs. imperative**: choosing the right style.

Comprehensions: Compact Collection Building

Key idea

A comprehension builds a collection from an existing sequence using a compact loop-like expression.

```
squares = []
for n in range(6):
    squares.append(n*n)
```

```
squares = [n*n for n in range(6)]
```

- Use comprehensions when the transformation is short and readable.
- Use a normal loop when the logic needs several steps.

List Comprehensions: Syntax and Conditions

```python
# transform every value
squares = [n*n for n in range(1, 6)]

# keep only some values
evens = [n for n in range(10) if n % 2 == 0]

# transform and filter at the same time
labels = [f"ID-{n}" for n in range(5) if n != 3]
```

Pattern

```
[expression for item in iterable if condition]
```

Nested List Comprehensions

Use with care

Nested comprehensions are compact, but they can become hard to read quickly.

```
# all coordinate pairs from 0..2 and 0..1
pairs = [(x, y) for x in range(3) for y in range(2)]
```

- Read it like nested loops: outer loop first, inner loop second.
- If students need to pause and decode it, a normal nested loop may be better.

Dictionary Comprehensions

```python
# number -> square
squares = {n: n*n for n in range(1, 6)}

# word -> length
words = ["AI", "Python", "data"]
lengths = {word: len(word) for word in words}
```

Pattern

```
{key_expression:  value_expression for item in iterable}
```

- Useful when each item naturally produces a key and a value.
- Keys must be unique; repeated keys overwrite earlier values.

Set Comprehensions

```python
names = ["Aisha", "Omar", "Ali", "Aisha"]
first_letters = {name[0] for name in names}

print(first_letters)
# {'A', 'O'}
```

When useful?

Use a set comprehension when you want unique results from a transformation.

Generator Expressions: Lazy Evaluation

Key idea

A generator expression produces values one at a time instead of building the whole collection immediately.

```
total = sum(n*n for n in range(1_000_000))
```

- This can save memory for large data.
- Good for feeding values into functions like `sum()`, `max()`, or `any()`.
- Use a list comprehension when you need to store or reuse the full list.

Comprehension or Explicit Loop?

Use a comprehension when...	Use a loop when...
The logic is one clear transformation.	The logic needs many steps.
The condition is short.	You need debugging prints or intermediate variables.
The result is a new collection.	You are changing existing state carefully.
It improves readability.	It becomes clever but difficult to understand.

Main principle

Readable code is better than compressed code.

Anti-Pattern: Modifying a List While Iterating

```python
numbers = [1, 2, 3, 4, 5]

# Avoid this pattern
for n in numbers:
    if n % 2 == 0:
        numbers.remove(n)
```

Why it is risky

Changing a list while looping over it can cause items to be skipped or processed incorrectly.

```python
# Better: build a new list
numbers = [n for n in numbers if n % 2 != 0]
```

map(): Applying a Function to Every Element

`map()` represents the idea of **transforming each item** in an iterable using the same function.

```
numbers = [1, 2, 3, 4]

squares = map(lambda x: x ** 2, numbers)
print(list(squares))   # [1, 4, 9, 16]
```

Key idea

Use this pattern when every item should be converted in the same way: numbers to squares, text to uppercase, temperatures to another unit, and so on.

Beginner note

A normal `for` loop is often easier to read at first. Learn the concept before worrying about shorter syntax.

filter(): Selecting Items by a Condition

`filter()` keeps only the items that pass a **predicate**: a function that returns `True` or `False`.

```
numbers = [3, -2, 0, 8, -5, 10]

positives = filter(lambda x: x > 0, numbers)
print(list(positives))  # [3, 8, 10]
```

Predicate

`lambda x: x > 0` asks a yes/no question about each item.

Result

Only items where the answer is `True` are kept.

zip(): Combining Multiple Iterables

`zip()` pairs items from multiple iterables by position.

```
names = ["Aisha", "Omar", "Lina"]
scores = [95, 88, 91]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

What happens?

The first name is paired with the first score, the second name with the second score, and so on.

Careful

If the iterables have different lengths, `zip()` stops when the shortest one ends.

enumerate(): Counting While Iterating

`enumerate()` gives you both the **index** and the **value** while looping.

```
names = ["Aisha", "Omar", "Lina"]

for index, name in enumerate(names):
    print(index, name)
```

Why use it?

Use it when you need the position of each item without manually creating a counter.

Custom start

```
for number, name in enumerate(names, start=1):
    print(number, name)
```

sorted() with a key Function

`sorted()` can use a **key function** to decide how items should be ordered.

```
words = ["AI", "programming", "data", "Python"]

print(sorted(words))
print(sorted(words, key=len))
```

Key idea

`key=len` means: sort the words by their length, not alphabetically.

```
students = [("Aisha", 95), ("Omar", 88), ("Lina", 91)]

# Sort by score using a small function
print(sorted(students, key=lambda item: item[1]))
```

Functional vs. Imperative Style

Python supports both **imperative** code and **functional-style** tools.

Imperative style

Step-by-step instructions.

```
result = []
for x in numbers:
    if x > 0:
        result.append(x * 2)
```

Functional style

Transform and filter data with functions.

```
result = map(lambda x: x * 2,
             filter(lambda x: x > 0, numbers))
result = list(result)
```

Main principle

Choose the version that is easiest to read. Compact code is not automatically better code.

Variable-Length Positional Arguments: *args

Sometimes a function should accept **any number of positional arguments**. In Python, the common convention is to collect them using *args.

```
def average(*args):
    total = 0
    for value in args:
        total += value
    return total / len(args)

print(average(10, 20, 30))
print(average(5, 8, 12, 15))
```

What happens?

- args becomes a tuple of the extra positional values.
- The function can loop over those values.
- Use this when the number of inputs is naturally flexible.

Beginner warning

Do not use *args just to avoid clear function design. Named parameters are usually better when the inputs are known.

Variable-Length Keyword Arguments: **kwargs

Sometimes a function should accept optional named information. In Python, the common convention is to collect extra keyword arguments using **kwargs.

```
def print_profile(**kwargs):
    for name, value in kwargs.items():
        print(name, "=", value)

print_profile(name="Aisha",
              field="AI",
              level="beginner")
```

What happens?

- kwargs stores extra named arguments.
- Each argument has a name and a value.
- This is useful for optional settings and configuration.

How to read it

*args answers: "How many positional values?" **kwargs answers: "Which named options were provided?"

Module 5
FILE I/O & EXCEPTION HANDLING

File I/O, exceptions, and error handling

Module Content

This module covers reading and writing files, and building programs that recover gracefully from runtime errors.

- **File I/O**: why it matters and opening files with `open()`.
- **Reading and writing**: text files, modes, and safe handling with `with`.
- **Binary files**: images and other non-text data.
- **Exceptions**: what they are and the exception hierarchy.
- **Handling errors**: `try`, `except`, `else`, and `finally`.
- **Raising exceptions**: signalling problems and custom exception classes.

Why File I/O Matters

Until now, many programs used values typed directly into code or entered by the user. Real programs often need to **store**, **load**, and **process** data from files.

Examples of input files

- grades in a text file
- experiment readings
- configuration files
- logs generated by a system

Examples of output files

- reports
- cleaned data
- saved results
- error logs

Key idea

File I/O lets a program communicate with the outside world beyond the screen and keyboard.

Opening Files with open()

To work with a file, Python needs a **file object**. The `open()` function creates that connection.

```
infile = open("input.txt", "r")    # read text
outfile = open("output.txt", "w")  # write text
logfile = open("log.txt", "a")     # append text
```

Mode	Meaning
`r`	read an existing text file
`w`	write a text file; replace existing content
`a`	append new text to the end of a file
`rb`	read binary data, such as images
`wb`	write binary data

Important warning

Opening a file in `"w"` mode clears the old content if the file already exists.

Reading from Files

Python gives several ways to read file content. Choose the method that matches the task.

```
file = open("story.txt", "r")

all_text = file.read()       # whole file
one_line = file.readline()   # one line
lines = file.readlines()     # list of lines

file.close()
```

When to use what

- `read()`: small file, all content needed.
- `readline()`: process one line at a time.
- `readlines()`: keep all lines as a collection.

Common pattern

```
for line in open("story.txt", "r"):
    print(line.strip())
```

Writing to Files

Writing stores text so it can be used later by another program, notebook, or person.

```
outfile = open("report.txt", "w")

outfile.write("Experiment report\n")
outfile.write("Accuracy: 0.92\n")

outfile.close()
```

```
lines = [
    "Aisha\n",
    "Omar\n",
    "Sara\n"
]

outfile = open("names.txt", "w")
outfile.writelines(lines)
outfile.close()
```

Checkpoint

`write()` does not automatically add a new line. Add "`n`" when you want the next output to appear on a new line.

The with Statement: Safe File Handling

A safer way to work with files is to use `with`. It automatically closes the file when the block finishes.

```
with open("input.txt", "r") as infile:
    for line in infile:
        print(line.strip())

with open("output.txt", "w") as outfile:
    outfile.write("Done\n")
```

Why this matters

- Files are closed even if the code inside the block fails.
- The code clearly shows where file access starts and ends.
- It avoids forgetting `close()`.

Beginner rule

Prefer `with open(...)` for file handling unless you have a special reason not to.

Binary Files: Images and Other Non-Text Data

Not all files are text. Images, audio, PDFs, and compressed files are stored as **binary data**.

```
with open("photo.jpg", "rb") as file:
    data = file.read()

print(type(data))  # bytes
print(len(data))   # number of bytes
```

Text vs. binary

- Text files store characters.
- Binary files store bytes.
- Use `rb` and `wb` for binary content.

Beginner warning

Do not read an image with normal `"r"` text mode. Use `"rb"` so Python treats it as bytes.

What Is an Exception?

An **exception** is Python's way of saying: "Something unusual happened, and normal execution cannot continue here."

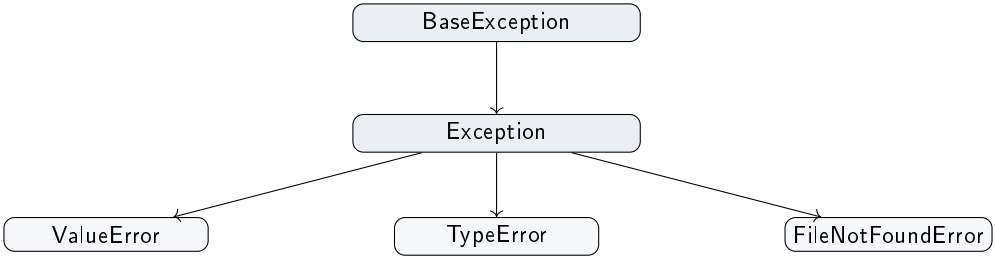
```
number = int("abc")       # ValueError
result = 10 / 0           # ZeroDivisionError
file = open("x.txt")      # FileNotFoundError if missing
```

Main idea

Exceptions separate **detecting a problem** from **deciding how to handle it**.

Exception Hierarchy: The Big Picture

Python exceptions are organized as classes. For most beginner programs, we catch subclasses of `Exception`.



Common examples

`ValueError`, `TypeError`, `ZeroDivisionError`, `IndexError`, `KeyError`, `FileNotFoundError`, and `PermissionError`.

The try and except Blocks

Put code that might fail inside a `try` block. Put the recovery plan inside an `except` block.

```
try:
    age = int(input("Age: "))
    print("Next year:", age + 1)
except ValueError:
    print("Please enter a whole number.")
```

How to read it

- Try to convert the input to an integer.
- If conversion fails, handle `ValueError`.
- The program can continue instead of crashing.

Handling File Errors Gracefully

File operations can fail. The file may not exist, the path may be wrong, or the user may not have permission.

```
from pathlib import Path

path = Path("data/results.txt")

try:
    text = path.read_text()
    print(text)
except FileNotFoundError:
    print("Could not find the file.")
except PermissionError:
    print("No permission to read this file.")
```

Key idea

Good programs do not crash immediately for expected file problems. They explain the problem and give the user a useful message.

else and finally

A complete exception structure can include `else` and `finally`.

```
try:
    value = int(input("Number: "))
except ValueError:
    print("Invalid number")
else:
    print("Square:", value ** 2)
finally:
    print("Finished checking input")
```

Block	Meaning
`try`	code that may raise an exception
`except`	code that handles a specific exception
`else`	runs only if no exception occurred
`finally`	always runs at the end

Catch Specific Exceptions

Avoid catching everything with a bare `except`. It can hide bugs and make debugging harder.

Better

```
try:
    value = int(text)
except ValueError:
    print("Not a number")
```

Avoid

```
try:
    value = int(text)
except:
    print("Something failed")
```

Rule of thumb

Catch the errors you expect and know how to handle. Let unexpected errors show up while you are developing.

Raising Exceptions

Sometimes a function detects invalid input and should report the problem to whoever called it.

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("amount exceeds balance")
    return balance - amount
```

Re-raising with context

```
try:
    amount = float(text)
except ValueError as error:
    raise ValueError("amount must be numeric") from error
```

Key idea

Use `raise` when your function cannot continue correctly and the caller should decide what to do.

Custom Exception Classes

For larger programs, you can define your own exception type. This makes errors more descriptive.

```
class InvalidGradeError(Exception):
    pass

def set_grade(grade):
    if grade < 0 or grade > 100:
        raise InvalidGradeError("grade must be between 0 and 100")
    return grade
```

When useful

Custom exceptions are useful when your program has domain-specific errors, such as invalid grades, invalid sensor readings, or invalid experiment settings.

Thank you!

Questions and discussion

Course takeaway

Reusable functions, rich data structures, and robust error handling are the building blocks of professional Python programs.